Integration of the FINS Protocol into the Open-Source HMI System FUXA

Denny Chrisnanda Frans Sebastian Gilang Hansagita

Bina Nusantara University

IEEE Conference Presentation

Outline

Motivation

- Omron PLCs dominate production lines at PT XYZ ($> 90\%$ machines).
- Factory Interface Network Service (FINS) is the de facto Omron protocol.
- Commercial HMI/SCADA systems are costly and vendor-locked.
- Open-source HMI FUXA lacks native FINS support.

Problem Statement

- Lack of native FINS support prevents FUXA adoption in Omron-centric plants.
- Middleware-based solutions increase latency and complexity.
- Alarm and real-time visualization reliability are critical requirements.

Research Questions

RQ1

How can FINS protocol support be implemented in FUXA with reliability and feature parity comparable to built-in protocols?

RQ2

How can stable, low-latency read/write operations be achieved for real-time visualization?

Main Contributions

- First native integration of Omron FINS into the FUXA HMI platform.
- Backend driver (`DeviceFins`) and frontend configuration support.
- Empirical evaluation of latency, reliability, resource usage, and alarms.
- Identification of alarm latency bottleneck in polling-based HMI systems.

Related Works

- Open-source SCADA using Node-RED, Grafana, InfluxDB.
- Web-based HMI platforms (FUXA, OpenPLC testbeds).
- Protocol integration studies: Modbus, OPC UA, DNP3.

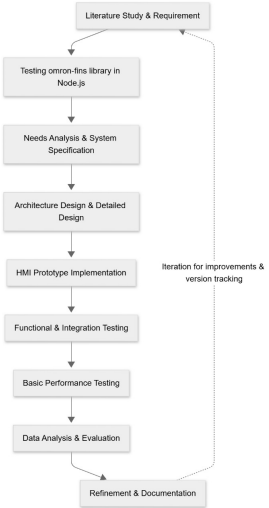
Research Gap

- Existing studies focus on Modbus, OPC UA, MQTT.
- No native open-source HMI support for Omron FINS found.
- Alarm latency and polling effects rarely analyzed explicitly.

Methodology Overview

- Iterative development approach:
  - Requirements analysis
  - System design
  - Implementation
  - Experimental evaluation

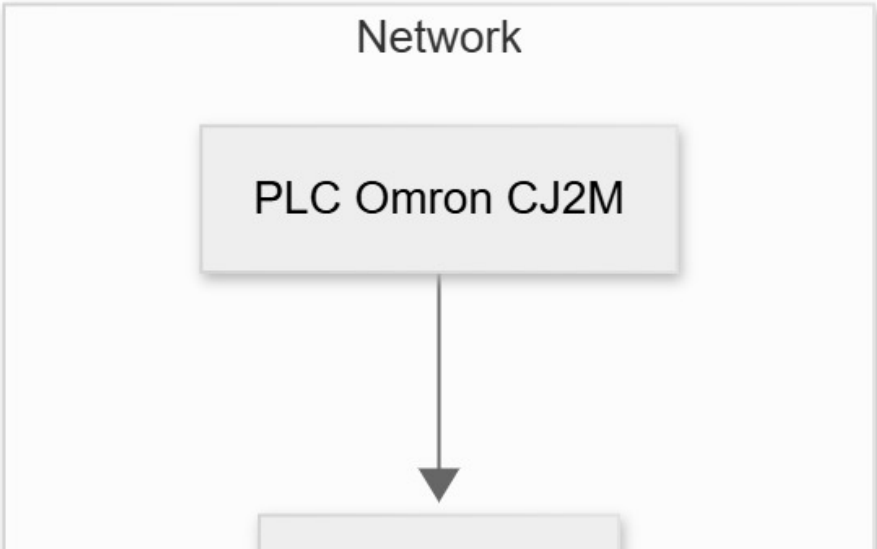
Research Workflow



System Architecture

- Three-layer architecture:
  - Omron PLC layer
  - Node.js backend (FUXA + DeviceFins)
  - Angular frontend (HMI visualization)

Architecture Diagram



Implementation

- `DeviceFins` driver in Node.js backend.
- Supports FINS over UDP and TCP.
- Read/write access to DM, CIO, and W memory areas.
- Automatic reconnection and timeout handling.

Frontend Integration

- Device configuration (IP, port, SA1, DA1).
- Tag editor for FINS-specific parameters.
- Real-time UI updates via event emitter.

Experimental Setup

- Omron CJ2M-CPU34 PLC.
- Node.js v18, FUXA v1.2.13.
- Stress test: 1000 reads, concurrency $= 10$.
- Metrics: latency, success rate, CPU, memory, alarms.

Evaluation Metrics

Metric	KPI Target
Latency	$\leq$200 ms
Success Rate	$\geq$95%
CPU Usage	$\leq$30%
Memory Usage	$\leq$500 MB
Alarm Detection	$\leq$200 ms
Startup Time	$\leq$10 s

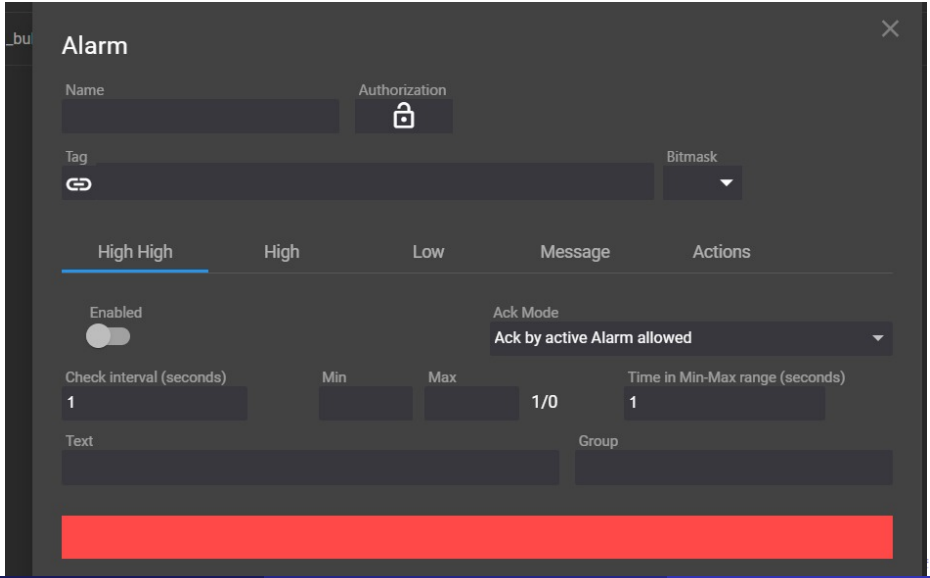
Performance Results

- Average read/write latency: **12 ms**.
- Communication success rate: **99%**.
- CPU usage: **2%**.
- Memory usage: **462 MB**.
- Startup time: **3.14 s**.

Alarm Detection Analysis

- Observed alarm detection delay: **1000 ms**.
- KPI target: 200 ms.
- Root cause: global 1-second polling interval in FUXA alarm engine.

Alarm Detection Interval



Comparison with Modbus/TCP

- Comparable latency (8–15 ms).
- Similar reliability under LAN conditions.
- Alarm delay identical due to shared polling-based alarm subsystem.

Conclusion

- First native FINS integration for the FUXA HMI.
- Reliable low-latency communication with Omron PLCs.
- Main limitation: polling-based alarm detection delay.

Future Work

- Event-driven or interrupt-based alarm handling.
- Enhanced security (TLS / VPN).
- Multi-PLC and large-scale deployment testing.

Thank You

Questions?